NTNU

TFE4141 Design of digital systems 1

Assignment 5 Examples

Author:

Oddbjørn H. Øvrelid

Max of 3

Implement an algorithm that reads 3 inputs and outputs the highest value.

$$output_N = Max(input_{3N+2}, input_{3N+1}, input_{3N}) \tag{1}$$

Task 1

```
1
2  uint input(void){
3      set_ready_in(1);
4      while(!valid_in)
5          wait();
6      uint val=read_bus_in();
7      set_ready_in(0);
8      return val;
9  }
10
11 void output(uint val){
12     write_bus_out(val);
13     set_valid_out(1);
14     while(!ready_out)
15         wait();
16     set_valid_out(0);
17 }
18
19 void main(void){
20     while(true){
21         uint reg1 = input();
22         uint reg2 = input();
23         uint reg3 = input();
24         if(reg1>reg2)
25             reg2 = reg1;
26         if(reg2>reg3)
27             reg3 = reg2;
28         output(reg3);
29     }
30 }
```

Figure 1: The psuedocode for reading 3 input values and outputting the maximum value. This is probably more detailed than necessary. It is ok to leave input and output as implicitly defined functions.

Task 2

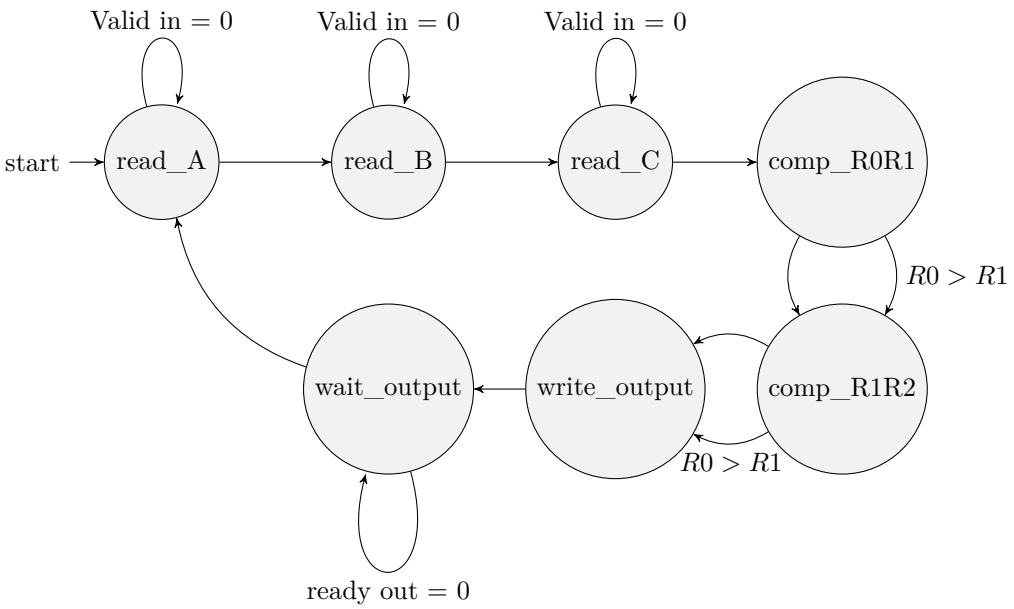


Figure 2: State diagram for the controller.

state	→	next state	condition	actions
read_A	→	read_A	valid in = 0	ready in 1
	→	read_B	valid in = 1	ready in 1 & write input to reg 0
read_B	→	read_B	valid in = 0	ready in 1
	→	read_C	valid in = 1	ready in 1 & write input to reg 1
read_C	→	read_C	valid in = 0	ready in 1
	→	comp_R0R1	valid in = 1	ready in 1 & write input to reg 2
comp_R0R1	→	comp_R1R2	$R0 > R1$	copy $R0 \rightarrow R1$
	→	comp_R1R2	$R0 \leq R1$	none
comp_R1R2	→	write_output	$R1 > R2$	copy $R1 \rightarrow R2$
	→	write_output	$R1 \leq R2$	none
write_output	→	wait_output	none	copy $R2 \rightarrow$ *output*
wait_output	→	wait_output	ready out = 0	valid out
	→	read_A	ready out = 1	valid out

Table 1: Transition table for statemachine.

Task 3

See files located under `/assignment_5/max_of_3/`.

Task 4

The implementations passes the testbench both in behavioural simulation and post synthesis simulation.

Sum of N integers

Implement an algorithm that sum a stream of numbers. The first number in the stream is the number of numbers in the stream. You can assume that you are never asked to create a sum of zero length.

$$sum(S) = \sum_{N=2}^{S_1+1} S_N \tag{2}$$

$$sum(\{3,1,2,3\}) = 1 + 2 + 3 = 6 \tag{3}$$

Task 1

```
1
2  uint input(void){
3      set_ready_in(1);
4      while(!valid_in)
5          wait();
6      uint val=read_bus_in();
7      set_ready_in(0);
8      return val;
9  }
10
11 void output(uint val){
12     write_bus_out(val);
13     set_valid_out(1);
14     while(!ready_out)
15         wait();
16     set_valid_out(0);
17 }
18
19 void main(void){
20     while(true){
21         uint N = input();
22         uint sum = 0;
23         for N-1 downto 0 {
24             sum = sum + input();
25         }
26         output(sum);
27     }
28 }
```

Figure 3: The psuedocode for summing a stream of numbers, with a count prefix. It is ok to leave input and output as implicitly defined functions.

```
1
2  void main(void){
3      loop{
4          uint N = input();
5          uint sum = 0;
6          loop{
7              if(N <= 1) goto done;
8              uint temp = input();
9              sum = sum + temp;
10             N=N-1;
11         }
12         done: output(sum);
13     }
14 }
```

Figure 4: Expanding the main function to get a better idea of what to do. as N is directly related to the flow control, it is decided to put it inside the controller. Note that the loop iterations is changed from $(N-1) \to 0$ to $N \to 1$. This is fine as we do not care about the value of N, just the number of iterations.

Task 2

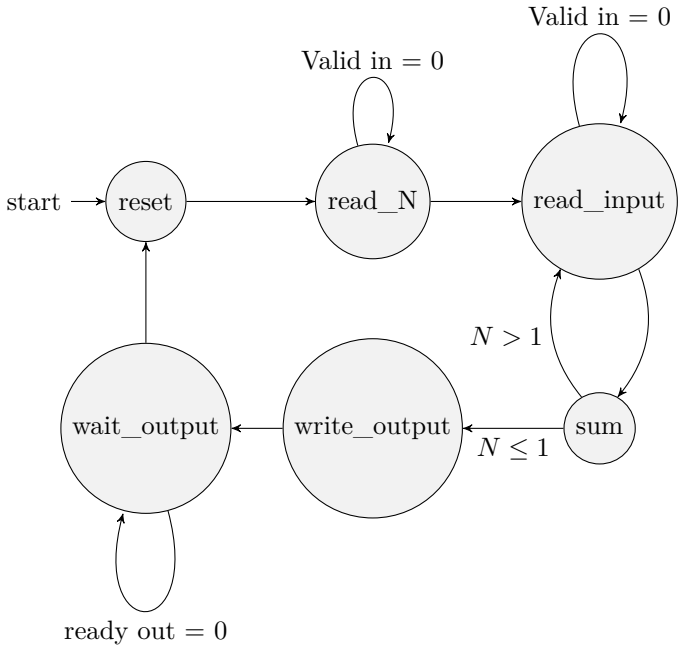


Figure 5: State diagram for the controller.

state	→	next state	condition	actions
reset	→	read_N	valid in = 0	$reg0 \leftarrow reg0 \oplus reg0$
read_N	→	read_N	valid in = 0	
	→	read_Input	valid in = 1	N $input$
read_input	→	read_input	valid in = 0	
	→	sum	valid in = 1	$reg1 \leftarrow input$
sum	→	read_input	$N > 1$	$reg0 \leftarrow reg0 + reg1$
	→	write_output	$N \leq 1$	$reg0 \leftarrow reg0 + reg1$ & $N \leftarrow N - 1$
write_output	→	wait_output	none	$output \leftarrow reg0$
wait_output	→	wait_output	ready out = 0	
	→	reset	ready out = 1	

Table 2: Transition table for statemachine.

Task 3

See files located under `/assignment_5/sum_of_n/`.

Task 4

The implementations passes the testbench both in behavioural simulation and post synthesis simulation.